Java Methodenliste

Autor

Luis Staudt

19. Dezember 2025

Inhaltsverzeichnis

Inhaltsverzeichnis 2
Grundlegende Methoden in Java 3
String-Methoden 3
ConsoleReader - Eingaben von der Konsole 5
readString() - String einlesen 6
readInt() - Ganzzahl einlesen 7
readDouble() - Kommazahl einlesen 8
readByte() - Sehr kleine Ganzzahl einlesen 9
readShort() - Kleine Ganzzahl einlesen 10
readLong() - Sehr große Ganzzahl einlesen 11
readFloat() - Einfache Kommazahl einlesen 12
readIntArray() - Ganzzahl-Array einlesen 13
readDoubleArray() - Kommazahl-Array einlesen 14
readStringArray() - String-Array einlesen 15
Weitere Array-Methoden 17
Häufige Fehler bei Strings 18

Grundlegende Methoden in Java

String-Methoden

length() - Länge eines Strings

Was macht die Methode?

Gibt die Anzahl der Zeichen im String zurück.

Syntax:

```
int laenge = text.length();
```

Beispiele:

```
String name = "Anna";
int laenge = name.length();  // laenge = 4

String wort = "Hallo Welt";
int laenge2 = wort.length();  // laenge2 = 10 (Leerzeichen zaehlt mit!)

String leer = "";
int laenge3 = leer.length();  // laenge3 = 0

String satz = "Java";
for (int i = 0; i < satz.length(); i++) {
    // Schleife laeuft 4 mal (von 0 bis 3)
}
```

Worauf achten:

- Die Zählung der Zeichen beginnt bei 1 (erster Buchstabe = Position 0, aber length = 1)
- Leerzeichen und Sonderzeichen zählen mit
- Bei Strings: `.length()` **mit Klammern**
- Bei Arrays: `.length` **ohne Klammern**

charAt(int index) - Zeichen an Position

Was macht die Methode?

Gibt das Zeichen an der angegebenen Position zurück. Die Zählung beginnt bei 0!

Syntax:

```
char zeichen = text.charAt(index);
```

Beispiele:

```
String wort = "Hallo";
char erstes = wort.charAt(0);   // erstes = 'H'
char zweites = wort.charAt(1);  // zweites = 'a'
char letztes = wort.charAt(4);  // letztes = 'o'

// Alle Zeichen durchlaufen
String text = "Java";
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    System.out.println(c);  // Gibt aus: J, a, v, a
}

// Pruefung einzelner Zeichen
String eingabe = "A1B2";
if (eingabe.charAt(0) == 'A') {
    System.out.println("Beginnt mit A");
}

// Zeichen vergleichen
String code = "12345";
if (code.charAt(2) == '3') {
    System.out.println("An Position 2 steht eine 3");
}
```

Worauf achten:

- Index beginnt bei 0
- Index muss zwischen 0 und `text.length() - 1` liegen
- Bei ungültigem Index: `StringIndexOutOfBoundsException`
- Gibt ein `char` zurück, keinen `String`

ConsoleReader - Eingaben von der Konsole

Der `ConsoleReader` ist eine Hilfsklasse zum einfachen Einlesen von Benutzereingaben. Die Klasse wird typischerweise mit statischem Import verwendet:

```
import static prog.ConsoleReader.*;
```

Alle Methoden haben zwei Varianten:

- Ohne Parameter: Liest direkt die Eingabe
- Mit String-Parameter: Zeigt zuerst einen Prompt (Aufforderungstext) an

readString() - String einlesen

Was macht die Methode?

Liest eine Zeile Text von der Konsole ein und gibt sie als String zurück.

Syntax:

```
String text = readString();                // Ohne Prompt
String text = readString("Eingabetext");   // Mit Prompt
```

Beispiele:

```
// Einfache Eingabe
String name = readString("Wie heisst du? ");
System.out.println("Hallo " + name);

// Mehrere Eingaben
String vorname = readString("Vorname");
String nachname = readString("Nachname");
String vollername = vorname + " " + nachname;

// Ohne Prompt
System.out.println("Gib einen Text ein:");
String eingabe = readString();

// In Schleife verwenden
String antwort;
do {
    antwort = readString("Weiter? (ja/nein)");
} while (!antwort.equals("nein"));
```

Worauf achten:

- Liest die gesamte Zeile bis Enter
- Gibt einen String zurück (auch bei Zahlen!)
- Für Zahlen: spezielle read-Methoden verwenden
- Bei leerem Input: leerer String , nicht `null`

readInt() - Ganzzahl einlesen

Was macht die Methode?

Liest eine Ganzzahl von der Konsole ein und gibt sie als `int` zurück.

Syntax:

```
int zahl = readInt();                        // Ohne Prompt
int zahl = readInt("Gib eine Zahl ein");     // Mit Prompt
```

Beispiele:

```
// Alter einlesen
int alter = readInt("Wie alt bist du? ");
System.out.println("Du bist " + alter + " Jahre alt");

// Berechnungen
int a = readInt("Erste Zahl");
int b = readInt("Zweite Zahl");
int summe = a + b;
System.out.println("Summe: " + summe);

// Array-Groesse einlesen
int anzahl = readInt("Wie viele Zahlen? ");
int[] zahlen = new int[anzahl];
for (int i = 0; i < zahlen.length; i++) {
    zahlen[i] = readInt("Zahl " + (i + 1));
}

// Schleifensteuerung
int wiederholungen = readInt("Wie oft wiederholen? ");
for (int i = 0; i < wiederholungen; i++) {
    System.out.println("Durchlauf " + (i + 1));
}
```

Worauf achten:

- Bei ungültiger Eingabe (z.B. Text): Programm bricht ab
- Nur Ganzzahlen im int-Bereich: -2.147.483.648 bis 2.147.483.647
- Für größere Zahlen: `readLong()` verwenden
- Für Kommazahlen: `readDouble()` verwenden

readDouble() - Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl von der Konsole ein und gibt sie als `double` zurück.

Syntax:

```
double zahl = readDouble();                    // Ohne Prompt
double zahl = readDouble("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Preis einlesen
double preis = readDouble("Preis in Euro");
double mehrwertsteuer = preis * 0.19;
double gesamtpreis = preis + mehrwertsteuer;
System.out.println("Gesamt: " + gesamtpreis + " Euro");

// Durchschnitt berechnen
double note1 = readDouble("Note 1");
double note2 = readDouble("Note 2");
double note3 = readDouble("Note 3");
double durchschnitt = (note1 + note2 + note3) / 3.0;
System.out.println("Durchschnitt: " + durchschnitt);

// Temperatur
double celsius = readDouble("Temperatur in Celsius");
double fahrenheit = celsius * 9.0 / 5.0 + 32.0;
System.out.println(celsius + " C  = " + fahrenheit + " F ");

// Auch Ganzzahlen moeglich
double wert = readDouble("Zahl");  // Eingabe: 5
// wert = 5.0
```

Worauf achten:

- Dezimaltrennzeichen: Komma `,` oder Punkt `.` (je nach System)
- Bei ungültiger Eingabe (z.B. Text): Programm bricht ab
- Kann auch Ganzzahlen einlesen (werden zu double konvertiert)
- Für einfache Kommazahlen: `readFloat()` verwenden

readByte() - Sehr kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr kleine Ganzzahl von der Konsole ein und gibt sie als `byte` zurück.

Syntax:

```
byte zahl = readByte();                    // Ohne Prompt
byte zahl = readByte("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Alter (wenn < 127)
byte alter = readByte("Alter");

// Prozent (0-100)
byte prozent = readByte("Prozent (0-100)");
```

Worauf achten:

- Wertebereich: nur -128 bis 127
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readShort() - Kleine Ganzzahl einlesen

Was macht die Methode?

Liest eine kleine Ganzzahl von der Konsole ein und gibt sie als `short` zurück.

Syntax:

```
short zahl = readShort();                    // Ohne Prompt
short zahl = readShort("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Jahr
short jahr = readShort("Geburtsjahr");

// Distanz in Metern
short meter = readShort("Distanz in Metern");
```

Worauf achten:

- Wertebereich: -32.768 bis 32.767
- Bei zu großen Zahlen: Fehler
- Meist wird `readInt()` bevorzugt

readLong() - Sehr große Ganzzahl einlesen

Was macht die Methode?

Liest eine sehr große Ganzzahl von der Konsole ein und gibt sie als `long` zurück.

Syntax:

```
long zahl = readLong();                    // Ohne Prompt
long zahl = readLong("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Sehr gro e Zahlen
long weltbevoelkerung = readLong("Weltbevoelkerung");

// Millisekunden
long millis = readLong("Millisekunden");
```

Worauf achten:

- Für sehr große Zahlen (größer als ca. 2 Milliarden)
- Wertebereich: ca. -9,2 $\times$ 10^{18} bis 9,2 $\times$ 10^{18}
- Für normale Zahlen: `readInt()` verwenden

readFloat() - Einfache Kommazahl einlesen

Was macht die Methode?

Liest eine Kommazahl mit einfacher Genauigkeit von der Konsole ein und gibt sie als `float` zurück.

Syntax:

```
float zahl = readFloat();                    // Ohne Prompt
float zahl = readFloat("Gib eine Zahl ein"); // Mit Prompt
```

Beispiele:

```
// Gewicht
float gewicht = readFloat("Gewicht in kg");

// Groesse
float groesse = readFloat("Groesse in m");
float bmi = gewicht / (groesse * groesse);
```

Worauf achten:

- Weniger genau als `double` (ca. 6-7 Dezimalstellen)
- Meist wird `readDouble()` bevorzugt
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)

readIntArray() - Ganzzahl-Array einlesen

Was macht die Methode?

Liest mehrere Ganzzahlen von der Konsole ein und gibt sie als `int[]`-Array zurück. Die Zahlen müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
int[] zahlen = readIntArray();                    // Ohne Prompt
int[] zahlen = readIntArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Mehrere Zahlen auf einmal einlesen
int[] noten = readIntArray("Noten (durch Leerzeichen getrennt)");
// Eingabe: 1 2 3 2 1
// noten = {1, 2, 3, 2, 1}

// Durchschnitt berechnen
int summe = 0;
for (int note : noten) {
    summe += note;
}
double durchschnitt = (double) summe / noten.length;
System.out.println("Durchschnitt: " + durchschnitt);

// Array direkt ausgeben
int[] zahlen = readIntArray("Zahlen");
System.out.println("Anzahl: " + zahlen.length);
for (int i = 0; i < zahlen.length; i++) {
    System.out.println("Zahl " + (i + 1) + ": " + zahlen[i]);
}
```

Worauf achten:

- Zahlen müssen durch **Leerzeichen** getrennt werden
- Array-Größe ergibt sich automatisch aus der Anzahl der Eingaben
- Bei ungültiger Eingabe (z.B. Text): Programm bricht ab
- Alle Zahlen müssen in einer Zeile eingegeben werden

readDoubleArray() - Kommazahl-Array einlesen

Was macht die Methode?

Liest mehrere Kommazahlen von der Konsole ein und gibt sie als `double[]`-Array zurück. Die Zahlen müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
double[] zahlen = readDoubleArray();                    // Ohne Prompt
double[] zahlen = readDoubleArray("Gib Zahlen ein");    // Mit Prompt
```

Beispiele:

```
// Preise einlesen
double[] preise = readDoubleArray("Preise");
// Eingabe: 9.99 14.50 7.25
// preise = {9.99, 14.50, 7.25}

// Gesamtpreis berechnen
double gesamt = 0.0;
for (double preis : preise) {
    gesamt += preis;
}
System.out.println("Gesamt: " + gesamt + " Euro");

// Temperaturwerte
double[] temps = readDoubleArray("Temperaturen");
double min = temps[0];
double max = temps[0];
for (double temp : temps) {
    if (temp < min) min = temp;
    if (temp > max) max = temp;
}
System.out.println("Min: " + min + ", Max: " + max);
```

Worauf achten:

- Zahlen müssen durch **Leerzeichen** getrennt werden
- Dezimaltrennzeichen: Komma oder Punkt (je nach System)
- Array-Größe ergibt sich automatisch aus der Anzahl der Eingaben
- Bei ungültiger Eingabe: Programm bricht ab

readStringArray() - String-Array einlesen

Was macht die Methode?

Liest mehrere Wörter von der Konsole ein und gibt sie als `String[]`-Array zurück. Die Wörter müssen durch Leerzeichen getrennt eingegeben werden.

Syntax:

```
String[] woerter = readStringArray();                    // Ohne Prompt
String[] woerter = readStringArray("Gib Woerter ein");   // Mit Prompt
```

Beispiele:

```
// Namen einlesen
String[] namen = readStringArray("Namen");
// Eingabe: Anna Bob Clara
// namen = {"Anna", "Bob", "Clara"}

// Alle Namen ausgeben
System.out.println("Teilnehmer:");
for (String name : namen) {
    System.out.println("- " + name);
}

// Woerter zaehlen
String[] woerter = readStringArray("Satz");
System.out.println("Anzahl Woerter: " + woerter.length);

// Laengstes Wort finden
String[] text = readStringArray("Text");
String laengstes = text[0];
for (String wort : text) {
    if (wort.length() > laengstes.length()) {
        laengstes = wort;
    }
}
System.out.println("Laengstes Wort: " + laengstes);
```

Worauf achten:

- Wörter müssen durch **Leerzeichen** getrennt werden
- Sätze mit Leerzeichen können nicht direkt eingelesen werden
- Array-Größe ergibt sich automatisch aus der Anzahl der Wörter
- Für Sätze: `readString()` verwenden und dann `.split()`

Weitere Array-Methoden

Der ConsoleReader bietet auch Methoden für andere Datentypen:

- `readByteArray()` - Liest `byte[]`-Array ein
- `readShortArray()` - Liest `short[]`-Array ein
- `readLongArray()` - Liest `long[]`-Array ein
- `readFloatArray()` - Liest `float[]`-Array ein

Diese funktionieren analog zu `readIntArray()` und `readDoubleArray()`.

Beispiel:

```
// Byte-Array
byte[] bytes = readByteArray("Bytes");

// Long-Array fuer grosse Zahlen
long[] ids = readLongArray("IDs");

// Float-Array
float[] gewichte = readFloatArray("Gewichte");
```

Häufige Fehler bei Strings

String.length ohne Klammern

```
// FALSCH
String text = "Hallo";
int laenge = text.length;  // Fehler: Strings brauchen ()

// RICHTIG
int laenge = text.length();    // String MIT Klammern
```

Index-Grenzen bei charAt

```
String wort = "Test";  // length = 4, Indizes: 0, 1, 2, 3
char c = wort.charAt(4);  // Fehler! Index 4 existiert nicht

// RICHTIG
char letztes = wort.charAt(wort.length() - 1);  // Index 3
char erstes = wort.charAt(0);                   // Index 0
```

char vs. String

```
String text = "A";
char buchstabe = 'A';

// FALSCH
if (text.charAt(0) == "A") { }  // Vergleicht char mit String

// RICHTIG
if (text.charAt(0) == 'A') { }  // char mit char vergleichen
```